Short Report: Data Structures and Performance in the Budget System

1. Data Structure Choices and Time Complexity

The Budget System was implemented using several efficient data structures to ensure fast data access and good scalability as the number of accounts and transactions increases.

The primary data structure used in the system is a **HashMap**, which stores account balances and spending categories as key–value pairs. In this structure, the key represents the account or category name, while the value represents the associated balance or total spending. Core operations such as `put()`, `get()`, and `remove()` execute in **average O(1)** time, allowing the program to quickly update balances or retrieve account data without performing a linear search through all entries.

A **HashSet** is used to maintain the list of locked accounts. Before processing withdrawals or transfers, the system checks whether the account is locked. The `contains()` operation in a HashSet runs in **O(1)** average time, enabling fast validation of account status regardless of the number of accounts stored.

The transaction history is maintained using an **ArrayList**, which records operations in chronological order. Appending new transactions to the end of an ArrayList has **amortized O(1)** time complexity. Although occasional resizing of the internal array may require **O(n)** time, the average cost of insertions remains constant over many operations.

Code Operation	Data Structure	Time Complexity	Explanation
`BudgetSystem.categorySpend.clear()`	HashMap	**O(n)**	Clearing a HashMap removes all stored entries. The operation must visit each element, so the time grows linearly with the number of stored categories.
`for (int i = 0; i < size; i++)`	Loop	**O(n)**	The loop runs exactly **size = n** times to create categories.
`categorySpend.put("Category"+i, i)`	HashMap	**O(1)** average per insertion	HashMap insertion is constant time on average due to hashing.
Filling the entire map	HashMap + loop	**O(n)** total	Since insertion happens **n times**, the total cost becomes linear.
`System.nanoTime()` (start)	System call	**O(1)**	Reading system time takes constant time.
`BudgetSystem.topCategories(5)`	HashMap processing	**O(n^2)**	The method sorts categories using **selection sort with nested loops**.
`System.nanoTime()` (end)	System call	**O(1)**	Getting the time again is constant.
`end - start`	Arithmetic	**O(1)**	Simple subtraction operation.
`System.out.println()`	Output	**O(1)**	Printing a single line takes constant time.

2. Manual Test Log

Test 1 – Creating an account
Input:
openAccount Savings
Output:
Account "Savings" created successfully.

```
Command: OPEN
Account: SAVINGS
Account created:🥳
```

Test 2 – Depositing money
Input:
deposit Savings 500
Output:
Deposit successful. New balance: 500

```
Command: DEPOSIT
Account: SAVINGS
Amount: 500
Deposit complete💚
```

Test 3 – Withdrawing money
Input:
withdraw Savings 200
Output:
Withdrawal successful. New balance: 300

```
Command: WITHDRAW
Account: SAVINGS
Amount: 200
Withdraw complete💚
```

Test 4 – Recording spending category
Input: spend Food 50
Output: spending recorded in category Food.

```
Command: SPEND
Account: SAVINGS
Category: FOOD
Amount: 50
Withdraw complete💚
```

Test 5 – Viewing transaction history
Input:
printLedger
Output:
1. Account created: Savings
2. Deposit: 500
3. Withdrawal: 200
4. Spending: Food – 50

```
Command: LEDGER
OPEN SAVINGS
DEPOSIT SAVINGS 500 KGS
WITHDRAW SAVINGS 200 KGS
WITHDRAW SAVINGS 50 KGS
SPEND SAVINGS FOOD 50 KGS
```

3. Empirical Timing Test #1: Top Categories Operation

We conducted an **empirical timing experiment** to evaluate the performance of the `topCategories()` method in the Budget System.

The purpose of this experiment was to measure how the execution time changes as the **number of categories (N)** increases. The `topCategories()` method identifies the categories with the highest spending amounts by sorting the categories according to their spending values.

The experiment helps verify how the algorithm behaves in practice and whether the observed results match the **theoretical time complexity**.

```java
static void runTest(int size) {  3 usages  esenaiz-gif

    // clear previous data
    BudgetSystem.categorySpend.clear();

    // fill categories
    for (int i = 0; i < size; i++) {
        BudgetSystem.categorySpend.put("Category" + i, i);
    }

    long start = System.nanoTime();

    BudgetSystem.topCategories(k: 5);

    long end = System.nanoTime();

    long time = end - start;

    System.out.println("Size: " + size + " → " + time + " ns");
```

```
===== TOP CATEGORIES Complexity Test =====

Category9 : 9 KGS
Category8 : 8 KGS
Category7 : 7 KGS
Category6 : 6 KGS
Category5 : 5 KGS
Size: 10 → 5792100 ns
Category999 : 999 KGS
Category998 : 998 KGS
Category997 : 997 KGS
Category996 : 996 KGS
Category995 : 995 KGS
Size: 1000 → 50780500 ns
Category9999 : 9999 KGS
Category9998 : 9998 KGS
Category9997 : 9997 KGS
Category9996 : 9996 KGS
Category9995 : 9995 KGS
Size: 10000 → 804322800 ns

===== TEST FINISHED =====
```

Experiment Setup

The following steps were used to run the experiment:

1. We created **N categories** with different spending values.
2. The categories were stored in a **HashMap<String, Integer>** called `categorySpend`.
3. Each category was assigned a spending value such as:
   a. `Category0 → 0 KGS`
   b. `Category1 → 1 KGS`
   c. `Category2 → 2 KGS`
4. The program then executed: `topCategories(5)`

to find the **top 5 categories with the highest spending**.

5. The execution time of the operation was measured using:

`System.nanoTime()`

6. The test was performed for **three different input sizes**:

```
testTopCategories(10)
testTopCategories(1000)
testTopCategories(1000000)
```

These tests represent **small, medium, and large datasets**.

Analysis of Results

The experimental results show that the **execution time increases rapidly as the number of categories increases**. This behavior occurs because the `topCategories()` method relies on **nested loops to sort the categories**, similar to the **selection sort algorithm**.

In this approach, the algorithm repeatedly scans the list of categories to find the next largest spending value. For each category, the algorithm compares it with many other categories to determine the correct ordering.

Because of this structure, the number of comparisons grows approximately as:

*n(n−1)/2n(n-1)/2*n(n−1)/2

This growth rate leads to a **quadratic time complexity**, which is expressed as:

*O(n2)O(n^2)*O(n2)

As the number of categories increases, the number of comparisons grows very quickly, causing the execution time to increase significantly.

For example, when the number of categories grows from **10 to 1000 and then to 1,000,000**, the runtime increases dramatically. This confirms the theoretical expectation that algorithms with **quadratic complexity scale poorly for large inputs**.

Conclusion

The empirical timing experiment confirms that the `topCategories()` method has **quadratic time complexity** ***O(n2)O(n^2)*****O(n2)** due to the use of **nested loops for sorting categories**. While the algorithm works efficiently for small datasets, its performance degrades significantly as the number of categories increases. This demonstrates a key limitation of quadratic algorithms when handling large amounts of data.

In contrast, operations based on data structures such as **HashMap or HashSet** typically run in **constant time** ***O(1)O(1)*****O(1)** on average, making them much more scalable for large systems.

Empirical Timing Test #2: Lock / Unlock Operation

To evaluate the performance of the **lock and unlock operations** in the Budget System, an empirical timing experiment was implemented using `System.nanoTime()`.

The purpose of this test was to observe how the execution time changes as the **number of accounts (N)** increases while performing repeated **lock and unlock operations** using a **HashSet**.

```java
        BudgetSystem.balanceByAccount.clear();
        BudgetSystem.lockedAccounts.clear();

        for (int i = 0; i < N; i++) {
            BudgetSystem.balanceByAccount.put("A" + i, 1000);
        }

        for (int r = 0; r < REPEATS; r++) {

            long start = System.nanoTime();

            for (int i = 0; i < N; i++) {
                BudgetSystem.lock( acc: "A" + i);
                BudgetSystem.unlock( acc: "A" + i);
            }

            long end = System.nanoTime();

            total += (end - start);
        }

        System.out.println("Size: " + N +
                " → lock/unlock avg time (HashSet O(1) avg): "
                + (total / REPEATS) + " ns");
    }
```

```
===== LOCK / UNLOCK Complexity Test =====

Size: 10 → lock/unlock avg time (HashSet O(1) avg): 64685 ns
Size: 1000 → lock/unlock avg time (HashSet O(1) avg): 3674242 ns
Size: 1000000 → lock/unlock avg time (HashSet O(1) avg): 170578514 ns

===== TEST FINISHED =====
```

```java
public class test2 {  esenaiz-gif

    static final int REPEATS = 7;  2 usages

    public static void main(String[] args) {  esenaiz-gif

        System.out.println("===== LOCK / UNLOCK Complexity Test =====\n");

        testLockUnlock( N: 10);        // small
        testLockUnlock( N: 1000);      // medium
        testLockUnlock( N: 1000000);   // large

        System.out.println("\n===== TEST FINISHED =====");
    }

    static void testLockUnlock(int N) {  3 usages  esenaiz-gif

        long total = 0;

        // prepare accounts
        BudgetSystem.balanceByAccount.clear();
        BudgetSystem.lockedAccounts.clear();

        for (int i = 0; i < N; i++) {
            BudgetSystem.balanceByAccount.put("A" + i, 1000);
        }

        for (int r = 0; r < REPEATS; r++) {

            long start = System.nanoTime();

            for (int i = 0; i < N; i++) {
                BudgetSystem.lock( acc: "A" + i);
```

Test Method

The program runs the `testLockUnlock(int N)` method with three different input sizes:

- **10 accounts**
- **1000 accounts**
- **1,000,000 accounts**

For each test:

1. The previous account data in `balanceByAccount` and `lockedAccounts` is cleared.
2. The program generates **N accounts** and stores them in a **HashMap** (`balanceByAccount`).
3. The test is repeated **7 times** (`REPEATS = 7`) to obtain a more stable average execution time.
4. For each repetition:
    a. The start time is recorded using `System.nanoTime()`.
    b. A loop runs through all **N accounts**.
    c. Each account is **locked** and then immediately **unlocked**.
5. The end time is recorded.
6. The elapsed time is added to a total time counter.
7. After all repetitions, the program calculates the **average execution time**.

The average time is calculated as:

```
average time = total time / REPEATS
```

Analysis of the Results

The execution time increases as the number of accounts grows. This happens because the program performs **N lock operations and N unlock operations** during each test.

Even though each individual **HashSet operation runs in average O(1)** time, the loop executes **N times**, which makes the overall complexity of the test approximately:

O(N)

This means the runtime increases **linearly** with the number of accounts.

The experimental results confirm this behavior: when the number of accounts increases from **10 → 1000 → 1,000,000**, the execution time also increases significantly.

Conclusion

The empirical timing experiment demonstrates that the **lock and unlock operations scale efficiently** when using a **HashSet**. Since both add() and remove() operations have **average constant time complexity**, the system can handle a large number of accounts without significant performance degradation.

However, because the test performs these operations **N times**, the total runtime grows **linearly with the input size**. This behavior is consistent with the theoretical complexity of the implemented algorithm.